

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
		Mr. Bingi Raju	
Course Code	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/I	Regulation	R23
Date and Day of Assignment	Week 2 - Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 3.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 3: Application for TGNPDCL – Electricity Bill Generation Using Python & AI Tools Lab Objectives - To design a real-world electricity billing application using Python - To use AI-assisted coding tools for logic generation and optimization - To understand conditional logic and arithmetic operations - To generate structured billing output similar to utility bills Lab Outcomes (LOs) After completing this lab, students will be able to:	Week2 - Wednesday	

	• Read and validate user input in Python • Apply conditional logic for tariff-based billing • Use AI tools to assist in program development • Calculate and display electricity bill components • Build a complete real-time application	
	Task 1: AI-Generated Logic for Reading Consumer Details Scenario An electricity billing system must collect accurate consumer data. Task Description Use an AI tool (GitHub Copilot / Gemini) to generate a Python program that: • Reads: ○ Previous Units (PU) ○ Current Units (CU) ○ Type of Customer • Calculates units consumed • Implements logic directly in the main program (no functions) Expected Output • Correct input reading • Units consumed calculation • Screenshot showing AI-generated code Sample input and output <pre>C:\Users\prade> OneDrive > Pradeep > log\logReading.py > ... 1 # AI-Generated Logic for Reading Consumer Details 2 # Electricity Billing System - Consumer Data Management 3 4 # ===== 5 # CONSUMER DETAILS READING AND CALCULATION 6 # ===== 7 8 print("-" * 50) 9 print("ELECTRICITY BILLING SYSTEM") 10 print("Consumer Details and Billing Calculation") 11 print("-" * 50) 12 print() 13 14 # Read consumer information 15 consumer_name = input("Enter Consumer Name: ") 16 consumer_id = input("Enter Consumer ID: ") 17 18 print("\n--- Enter Meter Readings ---") 19 20 # Read Previous Units 21 while True: 22 try: 23 previous_units = float(input("Enter Previous Units (PU): ")) 24 if previous_units < 0: 25 print("Error: Units cannot be negative. Please try again.") 26 continue 27 break 28 except ValueError: 29 print("Error: Please enter a valid number.") 30 31 # Read Current Units 32 while True: 33 try: 34 current_units = float(input("Enter Current Units (CU): ")) 35 if current_units < 0: 36 print("Error: Units cannot be negative. Please try again.") 37 continue 38 if current_units < previous_units: 39 print("Error: Current units cannot be less than previous units. Please try again.") 40 continue 41 break 42 except ValueError: 43 print("Error: Please enter a valid number.")</pre>	

```

C:\Users\prade > OneDrive > Pradeep > logicReading.py > ...
44
45 # Read Type of Customer
46 print("\n--- Select Customer Type ---")
47 print("1. Residential")
48 print("2. Commercial")
49 print("3. Industrial")
50
51 customer_type_choice = input("Enter customer type (1/2/3): ")
52
53 if customer_type_choice == "1":
54     customer_type = "Residential"
55     rate_per_unit = 5.50
56 elif customer_type_choice == "2":
57     customer_type = "Commercial"
58     rate_per_unit = 7.75
59 elif customer_type_choice == "3":
60     customer_type = "Industrial"
61     rate_per_unit = 4.25
62 else:
63     print("Invalid choice! Defaulting to Residential.")
64     customer_type = "Residential"
65     rate_per_unit = 5.50
66
67 # =====
68 # CALCULATION LOGIC (Direct Implementation)
69 # =====
70
71 # Calculate units consumed
72 units_consumed = current_units - previous_units
73
74 # Calculate total bill amount
75 total_bill = units_consumed * rate_per_unit
76
77 # Calculate tax (10% of bill)
78 tax = total_bill * 0.10
79 final_amount = total_bill + tax
80
81 # =====
82 # DISPLAY RESULTS
83 # =====
84
85 print("\n" + "=" * 50)
86 print("BILLING SUMMARY")
87 print("=" * 50)
88 print(f"Consumer Name      : {consumer_name}")
89
C:\Users\prade > OneDrive > Pradeep > logicReading.py > ...
89 print(f"Consumer ID      : {consumer_id}")
90 print(f"Customer Type      : {customer_type}")
91 print(f"Rate per Unit        : Rs. {rate_per_unit:.2f}")
92 print("=" * 50)
93 print(f"Previous Units (PU)   : {previous_units:.2f}")
94 print(f"Current Units (CU)    : {current_units:.2f}")
95 print(f"Units Consumed        : {units_consumed:.2f} units")
96 print("=" * 50)
97 print(f"Total Bill Amount     : Rs. {total_bill:.2f}")
98 print(f"Tax (10%)             : Rs. {tax:.2f}")
99 print(f"FINAL AMOUNT DUE      : Rs. {final_amount:.2f}")
100 print("=" * 50)
101
102 # Status based on consumption
103 if units_consumed > 100:
104     print("⚠ HIGH CONSUMPTION - Please check for leaks or wastage")
105 elif units_consumed > 50:
106     print("✓ MODERATE CONSUMPTION - Within normal range")
107 else:
108     print("✓ LOW CONSUMPTION - Excellent usage pattern")
109
110 print("=" * 50)
111

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER ⓘ

```

Consumer ID      : 112589
Customer Type    : Residential
Rate per Unit    : Rs. 5.50
-----
Previous Units (PU) : 52.20
Current Units (CU)  : 1555.25
Units Consumed     : 1503.05 units
-----
Total Bill Amount  : Rs. 8266.77
Tax (10%)         : Rs. 826.68
FINAL AMOUNT DUE  : Rs. 9093.45
-----
⚠ HIGH CONSUMPTION - Please check for leaks or wastage
-----

```

Description:

Electricity Billing System - Program Description:

This Python program automates consumer billing for an electricity company by reading Previous Units (PU), Current Units (CU), and Customer Type. It validates all inputs to prevent errors and calculates units consumed by subtracting PU from CU. The program applies tiered pricing based on customer category (Residential: ₹5.50, Commercial: ₹7.75, Industrial: ₹4.25 per unit) and automatically computes tax (10%) on the bill amount. Finally, it displays a comprehensive billing summary with consumption status alerts to help consumers identify unusual usage patterns and manage their electricity expenses efficiently.

Task 2: Energy Charges Calculation Based on Units Consumed

Scenario

Energy charges depend on the number of units consumed and customer type.

Task Description

Review the AI-generated code from Task 1 and extend it to:

- Calculate **Energy Charges (EC)**
- Use conditional statements based on:
 - Domestic
 - Commercial
 - Industrial consumers
- Improve readability using AI prompts such as:
 - *"Simplify energy charge calculation logic"*
 - *"Optimize conditional statements"*

Expected Output

- Correct EC calculation
- Clear conditional logic
- Original and improved versions (optional)
- Sample execution results

```
C:\Users\prade > OneDrive > Pradeep > calculation.py > ...
1  """
2  Energy Charges Calculation Based on Units Consumed
3  Calculates electricity charges using slab-based pricing for three customer types.
4  Supports Domestic, Commercial, and Industrial consumers with progressive rate structures.
5  Each customer type has different unit thresholds and rates per unit.
6  Uses optimized loop-based calculation for simplified and maintainable code.
7  """
8
9  # Rate structures for each customer type
10 RATES = {
11     'domestic': [(100, 5.0), (300, 7.0), (float('inf'), 10.0)],
12     'commercial': [(500, 8.0), (1000, 10.0), (float('inf'), 12.0)],
13     'industrial': [(1000, 6.0), (5000, 8.0), (float('inf'), 10.0)]
14 }
15
16 def calculate_energy_charges(units, customer_type):
17     """Calculate energy charges based on units and customer type."""
18     customer_type = customer_type.lower()
19
20     if customer_type not in RATES:
21         return ('error: f'Invalid customer type: {customer_type}')
22
23     charges = 0
24     prev_limit = 0
25
26     for limit, rate in RATES[customer_type]:
27         if units <= prev_limit:
28             break
29         slab_units = min(units, limit) - prev_limit
30         charges += slab_units * rate
31         prev_limit = limit
32
33     return {
34         'units': units,
35         'customer_type': customer_type,
36         'energy_charges': round(charges, 2)
37     }
```

```
38
39 # Test Cases
40 test_cases = [
41     (50, 'Domestic'), (150, 'Domestic'), (350, 'Domestic'),
42     (300, 'Commercial'), (750, 'Commercial'), (1500, 'Commercial'),
43     (800, 'Industrial'), (3000, 'Industrial'), (6000, 'Industrial')
44 ]
45
46 print("-" * 60)
47 print("ENERGY CHARGES CALCULATION")
48 print("-" * 60)
49
50 for units, ctype in test_cases:
51     result = calculate_energy_charges(units, ctype)
52     if 'error' not in result:
53         print(f"\n{result['customer_type'].upper()} | Units: {result['units']} | Charges: Rs. {result['energy_charges']:.2f}")
54     else:
55         print(f"\nError: {result['error']}")
56
57 print("\n" + "-" * 60)
58
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER ⓘ

=====

ENERGY CHARGES CALCULATION

=====

DOMESTIC | Units: 50 | Charges: Rs. 250.00

DOMESTIC | Units: 150 | Charges: Rs. 850.00

DOMESTIC | Units: 350 | Charges: Rs. 2400.00

COMMERCIAL | Units: 300 | Charges: Rs. 2400.00

COMMERCIAL | Units: 750 | Charges: Rs. 6500.00

COMMERCIAL | Units: 1500 | Charges: Rs. 15000.00

INDUSTRIAL | Units: 800 | Charges: Rs. 4800.00

INDUSTRIAL | Units: 3000 | Charges: Rs. 22000.00

INDUSTRIAL | Units: 6000 | Charges: Rs. 48000.00

=====

INDUSTRIAL | Units: 6000 | Charges: Rs. 48000.00

=====

INDUSTRIAL | Units: 6000 | Charges: Rs. 48000.00

=====

Description :

Energy Charges Calculation Based on Units Consumed

Calculates electricity charges using slab-based pricing for three customer types.

Supports Domestic, Commercial, and Industrial consumers with progressive rate structures.

Each customer type has different unit thresholds and rates per unit.

Uses optimized loop-based calculation for simplified and maintainable code.

Task 3: Modular Design Using AI Assistance (Using Functions)

Scenario

Billing logic must be reusable for multiple consumers.

Task Description

Use AI assistance to generate a Python program that:

- Uses user-defined functions to:
 - Calculate Energy Charges
 - Calculate Fixed Charges
- Returns calculated values
- Includes meaningful comments

Expected Output

- Function-based Python program
- Correct EC and FC values
- Screenshots of AI-assisted function generation
- Test cases with outputs

```

C:\Users\prade > OneDrive > Pradeep > twoprobmix.py > ...
1  """
2  Modular Billing System using Functions
3  """
4  This program demonstrates modular design for calculating:
5  1. Energy Charges (EC) - based on units consumed
6  2. Fixed Charges (FC) - based on customer type
7  Reusable functions for multiple consumers with clear separation of concerns.
8  """
9
10 # =====
11 # RATE CONFIGURATION - Centralized rates for all customer types
12 # =====
13
14 # Fixed charges per customer type (monthly)
15 FIXED_CHARGES = {
16     'domestic': 100,
17     'commercial': 500,
18     'industrial': 1000
19 }
20
21 # Energy charge rates (slab-based) for each customer type
22 ENERGY_RATES = {
23     'domestic': [(100, 5.0), (300, 7.0), (float('inf'), 10.0)],
24     'commercial': [(500, 8.0), (1000, 10.0), (float('inf'), 12.0)],
25     'industrial': [(1000, 6.0), (5000, 8.0), (float('inf'), 10.0)]
26 }
27
28 # =====
29 # FUNCTION 1: Calculate Fixed Charges (FC)
30 # =====
31
32 def calculate_fixed_charges(customer_type):
33     """
34     Calculate fixed charges based on customer type.
35
36     Args:
37         customer_type (str): 'Domestic', 'Commercial', or 'Industrial'
38
39     Returns:
40         float: Fixed charges for the customer type
41         None: If customer type is invalid
42     """
43     customer_type = customer_type.lower()
44
45     if customer_type not in FIXED_CHARGES:
46         print(f"Error: Invalid customer type '{customer_type}'")
47         return None
48
49     fc = FIXED_CHARGES[customer_type]
50     print(f"Fixed Charges ({customer_type}): Rs. {fc}")
51     return fc
52
53 # =====
54
55 C:\Users\prade > OneDrive > Pradeep > twoprobmix.py > ...
56 # =====
57 # FUNCTION 2: Calculate Energy Charges (EC)
58 # =====
59
60 def calculate_energy_charges(units, customer_type):
61     """
62     Calculate energy charges using slab-based pricing.
63
64     Args:
65         units (float): Number of units consumed
66         customer_type (str): 'Domestic', 'Commercial', or 'Industrial'
67
68     Returns:
69         float: Energy charges calculated based on slabs
70         None: If customer type is invalid
71     """
72     customer_type = customer_type.lower()
73
74     if customer_type not in ENERGY_RATES:
75         print(f"Error: Invalid customer type '{customer_type}'")
76         return None
77
78     # Calculate charges using slab-based pricing
79     charges = 0
80     prev_limit = 0
81
82     for limit, rate in ENERGY_RATES[customer_type]:
83         if units <= prev_limit:
84             break
85         slab_units = min(units, limit) - prev_limit
86         charges += slab_units * rate
87         prev_limit = limit
88
89     print(f"Energy Charges ({customer_type}, {units} units): Rs. {charges:.2f}")
90     return charges
91
92 # =====
93 # FUNCTION 3: Calculate Total Bill
94 # =====
95
96 def calculate_total_bill(units, customer_type):
97     """
98     Calculate total bill by combining EC and FC.
99
100     Args:
101         units (float): Number of units consumed
102         customer_type (str): 'Domestic', 'Commercial', or 'Industrial'
103
104     Returns:
105         dict: Complete billing breakdown with all charges
106     """
107     # Calculate individual charges
108     ec = calculate_energy_charges(units, customer_type)

```

```

100 ec = calculate_energy_charges(units, customer_type)
101 fc = calculate_fixed_charges(customer_type)
102
103 # Return None if either calculation failed
104 if ec is None or fc is None:
105     return None
106
107 # Calculate total
108 total_bill = ec + fc
109
110 return {
111     'customer_type': customer_type.lower(),
112     'units': units,
113     'energy_charges': round(ec, 2),
114     'fixed_charges': fc,
115     'total_bill': round(total_bill, 2)
116 }
117
118
119 # =====
120 # TEST CASES & SAMPLE EXECUTION
121 # =====
122
123 def print_bill_summary(bill):
124     """Print formatted bill summary."""
125     if bill is None:
126         return
127
128     print("\n" + "-" * 60)
129     print(f"Bill Summary - {bill['customer_type'].upper()} CUSTOMER")
130     print("-" * 60)
131     print(f"Units Consumed: {bill['units']} units")
132     print(f"Energy Charges (EC): Rs. {bill['energy_charges']:.2f}")
133     print(f"Fixed Charges (FC): Rs. {bill['fixed_charges']:.2f}")
134     print("-" * 60)
135     print(f"TOTAL BILL: Rs. {bill['total_bill']:.2f}")
136     print("-" * 60)
137
138
139 def main():
140     """Run test cases for billing system."""
141     # Test cases: (units, customer_type)
142     test_cases = [
143         (75, 'Domestic'),
144         (200, 'Domestic'),
145         (400, 'Domestic'),
146         (400, 'Commercial'),
147         (800, 'Commercial'),
148         (1500, 'Commercial'),
149         (600, 'Industrial'),
150         (3500, 'Industrial'),
151         (6500, 'Industrial')
152     ]
153
154     print("\n" * 40)
155     print("MODULAR BILLING SYSTEM - ENERGY CHARGES & FIXED CHARGES")
156     print("-" * 60)
157
158     # Process each test case
159     for units, customer_type in test_cases:
160         print(f">>> Processing: {units} units for {customer_type} customer")
161         bill = calculate_total_bill(units, customer_type)
162         print_bill_summary(bill)
163
164     print("\n" + "-" * 60)
165     print("TEST EXECUTION COMPLETED")
166     print("-" * 60)
167
168 if __name__ == "__main__":
169     main()
170

```

```

=====
MODULAR BILLING SYSTEM - ENERGY CHARGES & FIXED CHARGES
=====

```

```

>>> Processing: 75 units for Domestic customer
Energy Charges (domestic, 75 units): Rs. 375.00
Fixed Charges (domestic): Rs. 100

```

BILL SUMMARY - DOMESTIC CUSTOMER

Units Consumed:	75 units
Energy Charges (EC):	Rs. 375.00
Fixed Charges (FC):	Rs. 100.00
TOTAL BILL:	Rs. 475.00

```

>>> Processing: 200 units for Domestic customer
Energy Charges (domestic, 200 units): Rs. 1200.00
Fixed Charges (domestic): Rs. 100

```

BILL SUMMARY - DOMESTIC CUSTOMER

Units Consumed:	200 units
Energy Charges (EC):	Rs. 1200.00
Fixed Charges (FC):	Rs. 100.00
TOTAL BILL:	Rs. 1300.00

```

>>> Processing: 400 units for Domestic customer
Energy Charges (domestic, 400 units): Rs. 2000.00
Fixed Charges (domestic): Rs. 100

```

BILL SUMMARY - DOMESTIC CUSTOMER

Units Consumed:	400 units
Energy Charges (EC):	Rs. 2000.00
Fixed Charges (FC):	Rs. 100.00
TOTAL BILL:	Rs. 3000.00

```

>>> Processing: 400 units for Commercial customer
Energy Charges (commercial, 400 units): Rs. 3200.00
Fixed Charges (commercial): Rs. 500

```

BILL SUMMARY - COMMERCIAL CUSTOMER

Units Consumed:	400 units
Energy Charges (EC):	Rs. 3200.00
Fixed Charges (FC):	Rs. 500.00
TOTAL BILL:	Rs. 3700.00

```
>>> Processing: 800 units for Commercial customer
Energy Charges (commercial, 800 units): Rs. 7000.00
Fixed Charges (commercial): Rs. 500

BILL SUMMARY - COMMERCIAL CUSTOMER

Units Consumed:      800 units
Energy Charges (EC): Rs. 7000.00
Fixed Charges (FC):  Rs. 500.00

TOTAL BILL:          Rs. 7500.00

>>> Processing: 1500 units for Commercial customer
Energy Charges (commercial, 1500 units): Rs. 15000.00
Fixed Charges (commercial): Rs. 500

BILL SUMMARY - COMMERCIAL CUSTOMER

Units Consumed:      1500 units
Energy Charges (EC): Rs. 15000.00
Fixed Charges (FC):  Rs. 500.00

TOTAL BILL:          Rs. 15500.00

>>> Processing: 900 units for Industrial customer
Energy Charges (industrial, 900 units): Rs. 5400.00
Fixed Charges (industrial): Rs. 1000

BILL SUMMARY - INDUSTRIAL CUSTOMER

Units Consumed:      900 units
Energy Charges (EC):  Rs. 5400.00
Fixed Charges (FC):   Rs. 1000.00

TOTAL BILL:          Rs. 6400.00

>>> Processing: 3500 units for Industrial customer
Energy Charges (industrial, 3500 units): Rs. 26000.00
Fixed Charges (industrial): Rs. 1000

BILL SUMMARY - INDUSTRIAL CUSTOMER

Units Consumed:      3500 units
Energy Charges (EC):  Rs. 26000.00
Fixed Charges (FC):   Rs. 1000.00

TOTAL BILL:          Rs. 27000.00
```

- Description:
- Done! Added a concise 5-line description highlighting:
- **Line 1:** Program purpose (Modular Billing System)
 - **Line 2:** AI-assisted modular design overview
 - **Line 3:** Function 1 - Fixed Charges calculation
 - **Line 4:** Function 2 - Energy Charges with slab-based pricing
 - **Line 5:** Function 3 - Total Bill integration

Task 4: Calculation of Additional Charges
Scenario
Electricity bills include multiple additional charges.

- Task Description**
Extend the program to calculate:
- **FC** – Fixed Charges
 - **CC** – Customer Charges
 - **ED** – Electricity Duty (percentage of EC)

- Use AI prompts like:
- *“Add electricity duty calculation”*
 - *“Improve billing accuracy”*

- Expected Output**
- Individual charge values printed
 - Correct duty calculation
 - Well-structured output
 - Verified intermediate results


```

C:\Users\prade> OneDrive\Pradeep > additionalcharges.py > main
1
2
3 Calculation of Additional Charges - Comprehensive Billing System
4 EC - Energy Charges | FC - Fixed Charges | CC - Customer Charges | ED - Electricity Duty
5
6 # Configuration
7 FIXED_CHARGES = {'domestic': 100, 'commercial': 500, 'industrial': 1000}
8 CUSTOMER_CHARGES = {'domestic': 50, 'commercial': 200, 'industrial': 500}
9 ELECTRICITY_DUTY_PERCENT = {'domestic': 5, 'commercial': 8, 'industrial': 10}
10 ENERGY_RATES = {
11     'domestic': [(100, 5.0), (300, 7.0), (float('inf'), 10.0)],
12     'commercial': [(500, 8.0), (1000, 10.0), (float('inf'), 12.0)],
13     'industrial': [(1000, 6.0), (5000, 8.0), (float('inf'), 10.0)]
14 }
15
16 # =====
17 # CALCULATION FUNCTIONS
18 # =====
19
20 def calculate_energy_charges(units, customer_type):
21     """Calculate energy charges using slab-based pricing."""
22     customer_type = customer_type.lower()
23     if customer_type not in ENERGY_RATES:
24         return None
25     charges = 0
26     prev_limit = 0
27     for limit, rate in ENERGY_RATES[customer_type]:
28         if units <= prev_limit:
29             break
30         slab_units = min(units, limit) - prev_limit
31         charges += slab_units * rate
32         prev_limit = limit
33     return round(charges, 2)
34
35 def calculate_fixed_charges(customer_type):
36     """Calculate fixed monthly charges"""
37     customer_type = customer_type.lower()
38     return round(FIXED_CHARGES.get(customer_type, 0), 2)
39
40 def calculate_customer_charges(customer_type):
41     """Calculate customer-specific charges"""
42     customer_type = customer_type.lower()
43     return round(CUSTOMER_CHARGES.get(customer_type, 0), 2)
44
45 def calculate_electricity_duty(energy_charges, customer_type):
46     """Calculate electricity duty as percentage of energy charges"""
47     customer_type = customer_type.lower()
48     duty_percent = ELECTRICITY_DUTY_PERCENT.get(customer_type, 0)
49     return round((energy_charges * duty_percent) / 100, 2)
50
51 def calculate_total_bill(units, customer_type):
52     """Calculate complete bill with all charges: EC, FC, CC, ED"""
53     customer_type_lower = customer_type.lower()
54     if customer_type_lower not in ENERGY_RATES:
55         return None
56
57     ec = calculate_energy_charges(units, customer_type)
58     fc = calculate_fixed_charges(customer_type)
59     cc = calculate_customer_charges(customer_type)
60     ed = calculate_electricity_duty(ec, customer_type)
61
62     if ec is None:
63         return None
64
65     return {
66         'customer_type': customer_type_lower,
67         'units': units,
68         'EC': ec,
69         'FC': fc,
70         'CC': cc,
71         'ED': ed,
72         'total_bill': round(ec + fc + cc + ed, 2)
73     }
74
75 # =====
76 # DISPLAY FUNCTIONS
77 # =====
78
79 def print_intermediate_results(units, customer_type):
80     """Print intermediate calculation results for verification"""
81     print(f"\n [Intermediate Results - {units} units | {customer_type}]")
82     ec = calculate_energy_charges(units, customer_type)
83     print(f"    EC (Energy Charges):      Rs. {ec:.2f}")
84     fc = calculate_fixed_charges(customer_type)
85     print(f"    FC (Fixed Charges):       Rs. {fc:.2f}")
86     cc = calculate_customer_charges(customer_type)
87     print(f"    CC (Customer Charges):    Rs. {cc:.2f}")
88     ed = calculate_electricity_duty(ec, customer_type)
89     print(f"    ED (Electricity Duty):    Rs. {ed:.2f}")
90
91 def print_bill_summary(bill):
92     """Print formatted complete billing summary"""
93     if bill is None:
94         return
95     print("\n" + "-" * 60)
96     print(f"Bill - {bill['customer_type'].upper()} | Units: {bill['units']}")
97     print("-" * 60)
98     print(f"Energy Charges (EC):      Rs. {bill['EC']:.2f}")
99     print(f"Fixed Charges (FC):       Rs. {bill['FC']:.2f}")
100    print(f"Customer Charges (CC):    Rs. {bill['CC']:.2f}")
101    print(f"Electricity Duty (ED):    Rs. {bill['ED']:.2f}")
102    print("-" * 60)
103    print(f"TOTAL Bill:              Rs. {bill['total_bill']:.2f}")
104    print("-" * 60)
105
106 # =====
107 # TEST CASES
108 # =====
109
110 def main():
111     """Run test cases with comprehensive billing calculations"""
112     test_cases = [
113         (50, 'Domestic'), (200, 'Domestic'), (400, 'Domestic'),
114         (300, 'Commercial'), (900, 'Commercial'), (1500, 'Commercial'),
115         (800, 'Industrial'), (1500, 'Industrial'), (6000, 'Industrial')
116     ]
117
118     print("-" * 60)
119     print("ELECTRICITY BILLING SYSTEM - ADDITIONAL CHARGES CALCULATION")
120     print("-" * 60)
121

```

```

121
122     for units, customer_type in test_cases:
123         print(f"\n>>> {units} units | {customer_type} Customer")
124         print_intermediate_results(units, customer_type)
125         bill = calculate_total_bill(units, customer_type)
126         print_bill_summary(bill)
127
128     print("\n" + "=" * 60)
129     print("TEST EXECUTION COMPLETED")
130     print("=" * 60)
131
132 if __name__ == '__main__':
133     main()
134

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER

***** ELECTRICITY BILLING SYSTEM - ADDITIONAL CHARGES CALCULATION *****

>>> 50 units | Domestic Customer

```

[Intermediate Results - 50 units | Domestic]
EC (Energy Charges):    Rs. 250.00
FC (Fixed Charges):     Rs. 100.00
CC (Customer Charges):  Rs. 50.00
ED (Electricity Duty):  Rs. 12.50

```

***** BILL - DOMESTIC | Units: 50 *****

```

Energy Charges (EC):    Rs. 250.00
Fixed Charges (FC):     Rs. 100.00
Customer Charges (CC):  Rs. 50.00
Electricity Duty (ED):  Rs. 12.50

```

```

-----
TOTAL BILL:              Rs. 412.50
-----

```

>>> 200 units | Domestic Customer

```

[Intermediate Results - 200 units | Domestic]
EC (Energy Charges):    Rs. 1200.00
FC (Fixed Charges):     Rs. 100.00
CC (Customer Charges):  Rs. 50.00
ED (Electricity Duty):  Rs. 60.00

```

***** BILL - DOMESTIC | Units: 200 *****

```

Energy Charges (EC):    Rs. 1200.00
Fixed Charges (FC):     Rs. 100.00
Customer Charges (CC):  Rs. 50.00
Electricity Duty (ED):  Rs. 60.00

```

```

-----
TOTAL BILL:              Rs. 1410.00
-----

```

>>> 400 units | Domestic Customer

```

[Intermediate Results - 400 units | Domestic]
EC (Energy Charges):    Rs. 2000.00
FC (Fixed Charges):     Rs. 100.00
CC (Customer Charges):  Rs. 50.00
ED (Electricity Duty):  Rs. 145.00

```

```
=====
BILL - DOMESTIC | Units: 400
=====
Energy Charges (EC):      Rs. 2900.00
Fixed Charges (FC):       Rs. 100.00
Customer Charges (CC):    Rs. 50.00
Electricity Duty (ED):    Rs. 145.00
=====
TOTAL BILL:              Rs. 3195.00
=====

>>> 300 units | Commercial Customer

[Intermediate Results - 300 units | Commercial]
EC (Energy Charges):      Rs. 2400.00
FC (Fixed Charges):       Rs. 500.00
CC (Customer Charges):    Rs. 200.00
ED (Electricity Duty):    Rs. 192.00
=====

BILL - COMMERCIAL | Units: 300
=====
Energy Charges (EC):      Rs. 2400.00
Fixed Charges (FC):       Rs. 500.00
Customer Charges (CC):    Rs. 200.00
Electricity Duty (ED):    Rs. 192.00
=====
TOTAL BILL:              Rs. 3292.00
=====

>>> 900 units | Commercial Customer

[Intermediate Results - 900 units | Commercial]
EC (Energy Charges):      Rs. 8000.00
FC (Fixed Charges):       Rs. 500.00
CC (Customer Charges):    Rs. 200.00
ED (Electricity Duty):    Rs. 640.00
=====

BILL - COMMERCIAL | Units: 900
=====
Energy Charges (EC):      Rs. 8000.00
Fixed Charges (FC):       Rs. 500.00
Customer Charges (CC):    Rs. 200.00
Electricity Duty (ED):    Rs. 640.00
=====
TOTAL BILL:              Rs. 9340.00
=====

>>> 1500 units | Commercial Customer

[Intermediate Results - 1500 units | Commercial]
EC (Energy Charges):      Rs. 15000.00
FC (Fixed Charges):       Rs. 500.00
CC (Customer Charges):    Rs. 200.00
ED (Electricity Duty):    Rs. 1200.00
=====

BILL - COMMERCIAL | Units: 1500
=====
Energy Charges (EC):      Rs. 15000.00
Fixed Charges (FC):       Rs. 500.00
Customer Charges (CC):    Rs. 200.00
Electricity Duty (ED):    Rs. 1200.00
=====
TOTAL BILL:              Rs. 16900.00
=====
```

- Description:
- Done! Added a detailed 5-line description covering:
- **Line 1:** Program purpose
 - **Line 2:** Overview of billing components
 - **Line 3:** EC calculation details
 - **Line 4:** FC and CC calculation details
 - **Line 5:** ED calculation details

Task 5: Final Bill Generation and Output Analysis
Scenario

The final electricity bill must present all values clearly.

Task Description

Develop the final Python application to:

- Calculate total bill:
- Total Bill = EC + FC + CC + ED
- Display:
 - Energy Charges (EC)
 - Fixed Charges (FC)
 - Customer Charges (CC)
 - Electricity Duty (ED)
 - Total Bill Amount
- Analyze the program based on:
 - Accuracy
 - Readability
 - Real-world applicability

Expected Output

- Complete electricity bill output
- Neatly formatted display

- Sample input/output
- Short analysis paragraph

```
C:\Users> grade > OneDrive > Pradeep > finalBil.py > ...
2 Electricity Bill Generation System
3 Purpose: Calculate and display final electricity bills with detailed breakdown
4 ***
5
6 def calculate_energy_charges(units, rate_per_unit):
7     """Calculate energy charges based on units consumed."""
8     return units * rate_per_unit
9
10
11 def calculate_fixed_charges(monthly_fixed_rate):
12     """Calculate fixed monthly charges."""
13     return monthly_fixed_rate
14
15
16 def calculate_customer_charges(customer_rate):
17     """Calculate customer service charges."""
18     return customer_rate
19
20
21 def calculate_electricity_duty(subtotal, duty_percentage):
22     """Calculate electricity duty based on percentage of subtotal."""
23     return (subtotal * duty_percentage) / 100
24
25
26
27 def calculate_total_bill(ec, fc, cc, ed):
28     """Calculate total bill amount."""
29     return ec + fc + cc + ed
30
31
32 def display_bill(customer_name, account_number, billing_month,
33                 units, ec, fc, cc, ed, total_bill):
34     """Display formatted electricity bill."""
35     print("\n" + "-"*60)
36     print(" *15 + "ELECTRICITY BILL")
37     print("-"*60)
38     print(f"Customer Name:      {customer_name}")
39     print(f"Account Number:      {account_number}")
40     print(f"Billing Period:      {billing_month}")
41     print(f"Units Consumed:      {units} kWh")
42     print("\n" + "-"*60)
43     print("BILL BREAKDOWN")
44     print("-"*60)
45     print(f"Energy Charges (EC):      Rs. {ec:>12,.2f}")
46     print(f"Fixed Charges (FC):      Rs. {fc:>12,.2f}")
47     print(f"Customer Charges (CC):    Rs. {cc:>12,.2f}")
48     print(f"Sub Total:              Rs. {(ec+fc+cc):>12,.2f}")
49     print(f"Electricity Duty (ED):    Rs. {ed:>12,.2f}")
50     print("-"*60)
51     print(f"TOTAL BILL AMOUNT:<40) Rs. {total_bill:>12,.2f}")
52     print("-"*60 + "\n")
53
54 C:\Users> grade > OneDrive > Pradeep > finalBil.py > ...
55
56 def main():
57     """Main function to run the electricity bill generation system."""
58
59     print("\n" + "-"*60)
60     print(" *10 + "ELECTRICITY BILL GENERATION SYSTEM")
61     print("-"*60)
62
63     # Get customer information
64     print("\n--- CUSTOMER INFORMATION ---")
65     customer_name = input("Enter Customer Name: ").strip()
66     account_number = input("Enter Account Number: ").strip()
67     billing_month = input("Enter Billing Month/Period (e.g., January 2026): ").strip()
68
69     # Get consumption and rate details
70     print("\n--- CONSUMPTION & RATE DETAILS ---")
71     try:
72         units = float(input("Enter Units Consumed (kWh): "))
73         rate_per_unit = float(input("Enter Rate per Unit (Rs.): "))
74         monthly_fixed_rate = float(input("Enter Fixed Monthly Charges (Rs.): "))
75         customer_rate = float(input("Enter Customer Service Charges (Rs.): "))
76         duty_percentage = float(input("Enter Electricity Duty Percentage (%): "))
77
78         # Validate inputs
79         if units < 0 or rate_per_unit < 0 or monthly_fixed_rate < 0 or customer_rate < 0 or duty_percentage < 0:
80             print("\nError: All values must be non-negative!")
81             return
82
83         # Calculate charges
84         ec = calculate_energy_charges(units, rate_per_unit)
85         fc = calculate_fixed_charges(monthly_fixed_rate)
86         cc = calculate_customer_charges(customer_rate)
87         subtotal = ec + fc + cc
88         ed = calculate_electricity_duty(subtotal, duty_percentage)
89         total_bill = calculate_total_bill(ec, fc, cc, ed)
90
91         # Display the bill
92         display_bill(customer_name, account_number, billing_month,
93                     units, ec, fc, cc, ed, total_bill)
94
95         # Display summary statistics
96         print("\n--- BILL SUMMARY ---")
97         print(f"Average Cost per Unit: Rs. {total_bill/units:.2f} (including all charges)")
98         print(f"Percentage Breakdown:")
99         print(f"Energy Charges:      {(ec/total_bill)*100:.1f}%")
100        print(f"Fixed Charges:      {(fc/total_bill)*100:.1f}%")
101        print(f"Customer Charges:    {(cc/total_bill)*100:.1f}%")
102        print(f"Electricity Duty:    {(ed/total_bill)*100:.1f}%")
103        print()
104
105    except ValueError:
106        print("\nError: Please enter valid numeric values!")
107        return
108
109 if __name__ == "__main__":
110     main()
111
```

```

=====
ELECTRICITY BILL GENERATION SYSTEM
=====

--- CUSTOMER INFORMATION ---
Enter Customer Name: xyz
Enter Account Number: 18824
Enter Billing Month/Period (e.g., January 2026): jan 2026

--- CONSUMPTION & RATE DETAILS ---
Enter Units Consumed (kWh): 110
Enter Rate per Unit (Rs.): 4.5
Enter Fixed Monthly Charges (Rs.): 150
Enter Customer Service Charges (Rs.): 50
Enter Electricity Duty Percentage (%): 5

=====
ELECTRICITY BILL
=====

Customer Name:      xyz
Account Number:     18824
Billing Period:      jan 2026
Units Consumed:      110.0 kWh

=====
BILL BREAKDOWN
=====

Energy Charges (EC):      Rs.    935.00
Fixed Charges (FC):        Rs.    150.00
Customer Charges (CC):     Rs.     50.00
Sub Total:                 Rs.   1,135.00
Electricity Duty (ED):     Rs.     56.75

TOTAL BILL AMOUNT                      Rs.   1,191.75

=====
--- BILL SUMMARY ---
Average Cost per Unit: Rs. 10.83 (including all charges)
Percentage Breakdown:
Energy Charges:         78.5%
Customer Charges (CC):   Rs.     50.00
Sub Total:               Rs.   1,135.00
Electricity Duty (ED):   Rs.     56.75

TOTAL BILL AMOUNT                      Rs.   1,191.75

=====
--- BILL SUMMARY ---
Average Cost per Unit: Rs. 10.83 (including all charges)
Percentage Breakdown:
Energy Charges:         78.5%

TOTAL BILL AMOUNT                      Rs.   1,191.75

=====
--- BILL SUMMARY ---
Average Cost per Unit: Rs. 10.83 (including all charges)
Percentage Breakdown:
Energy Charges:         78.5%
Customer Charges (CC):   Rs.     50.00
Sub Total:               Rs.   1,135.00
Electricity Duty (ED):   Rs.     56.75

TOTAL BILL AMOUNT                      Rs.   1,191.75

=====
--- BILL SUMMARY ---
Average Cost per Unit: Rs. 10.83 (including all charges)
Percentage Breakdown:
Energy Charges:         78.5%

TOTAL BILL AMOUNT                      Rs.   1,191.75

=====
--- BILL SUMMARY ---
Average Cost per Unit: Rs. 10.83 (including all charges)
Percentage Breakdown:
Energy Charges:         78.5%

TOTAL BILL AMOUNT                      Rs.   1,191.75

=====
--- BILL SUMMARY ---
Average Cost per Unit: Rs. 10.83 (including all charges)
Percentage Breakdown:
Energy Charges:         78.5%
Fixed Charges:           12.6%
Average Cost per Unit:   Rs. 10.83 (including all charges)
Percentage Breakdown:
Energy Charges:         78.5%
Customer Charges:        4.2%
Fixed Charges:           12.6%
Customer Charges:        4.2%
Fixed Charges:           12.6%
Customer Charges:        4.2%
Electricity Duty:        4.8%
Customer Charges:        4.2%
Electricity Duty:        4.8%
Electricity Duty:        4.8%

```

Description:

1. **Accuracy** - Validates inputs and uses correct billing formula
2. **Readability** - Well-structured code with clear function names and documentation
3. **Real-world applicability** - Mirrors actual billing systems with all standard charges and provides insights for customer understanding

The program is now complete with the full bill generation system including:

- ☒ Energy Charges (EC) calculation
- ☒ Fixed Charges (FC) calculation

	• ☒ Customer Charges (CC) calculation • ☒ Electricity Duty (ED) calculation • ☒ Total Bill = EC + FC + CC + ED • ☒ Neatly formatted bill display • ☒ Input validation • ☒ Summary statistics with percentage breakdown • ☒ Comprehensive 5-line analysis Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.	
--	---	--